

Software Engineering Practice Hw6

資電三 112504009 林紘喬

一、CDC-GK 的 3 個缺點分析

1. 變數覆蓋錯誤

- 程式碼位置：Line 167–168
- 問題：在處理金幣與銀幣的邏輯時，程式連續兩行對同一個變數 `inputgold[q]` 進行賦值。

```
inputgold[q] = Characters[i].gold % 10;  
inputgold[q] = Characters[i].silver % 10; // 覆蓋上一行內容
```
- 影響：導致 `inputgold` 陣列永遠只會存放銀礦的數值，金礦數據遺失。
- 修復：將第 168 行改為 `inputsilver[q] = Characters[i].silver % 10;`

2. 數量限制邏輯錯誤 (計數器不一致)

- 程式碼位置：Line 93–98
- 問題：在檢查「均富 (`rand == 8`)」道具的生成數量限制時，條件判斷誤用了金幣計數器 `i3`，但更新時卻是增加 `i8`。

```
if ( rand == 8 && i3 > 5 ) // 誤用 i3 判斷均富上限  
{  
    k--; continue;  
}  
else if ( rand == 8 && i8 <= 5 )  
    i8++;
```
- 影響：均富道具的產量將受到金幣數量的干擾，導致邏輯失控。
- 修復：應將判斷條件統一修改為 `i8 > 5`。

3. 潛在的空指標風險

- 程式碼位置：Line 150–156

- **問題**：程式在執行 `Characters[i] != null` 檢查之前，就已經先存取了該物件的成員變數（`x`, `y`, `gold`）。

```
tempx = Characters[i].x;    // line 150 ← 已存取成員
tempy = Characters[i].y;    // line 151
tempgold = Characters[i].gold; // line 152
...
if (Characters[i] != null) // line 156 ← 這時才檢查 null
```

- **影響**：如果 `Characters[i]` 為 `null`，程式會在第 150 行直接當機，後續的 `null` 檢查形同虛設。
- **修復**：將 `if (Characters[i] != null)` 檢查移至所有存取動作之前。

二、CDC-DJ 的 3 個缺點分析

1. 重複且不必要的 `import java.lang.*`

- **程式碼位置**：Line 8–9
- **問題**：重複導入 `java.lang.*`。事實上，Java 會預設載入該套件，不需額外撰寫。

```
import java.lang.*;
import java.lang.*;
```

2. 地圖障礙物初始化使用大量重複 code

- **程式碼位置**：Line 73–104
- **問題**：地圖障礙物使用大量的手動賦值（如 `map[0][13] = -1;`），重複了約 30 行。

```
map[0][13] = map[0][14] = map[0][15] = map[0][16] = -1;
map[1][13] = map[1][14] = map[1][15] = map[1][16] = -1;
map[2][13] = map[2][14] = map[2][15] = map[2][16] = -1;
// ... (同樣模式重複十幾次)
```

- **影響**：程式碼冗長且難以維護。若要更換地圖佈局，修改成本極高。
- **修復**：應使用雙層 `for` 迴圈來進行區域初始化。

3. 碰撞偵測迴圈自行設定上限 4，而非使用 `characterNumber` 常數

- 程式碼位置：Line 323
- 問題：碰撞偵測迴圈上限被寫死為 4，即便程式上方已定義了常數 `characterNumber = 6`。

```
for (int z = 1; z <= 4; z++) { //去看其他腳色是否有碰撞的情況
```

- 影響：當角色數量增加到 5 或 6 人時，系統將無法偵測到這些新角色的碰撞，產生邏輯錯誤。
- 修復：將迴圈條件改為 `z <= characterNumber`。

三、DOM-GK 最佳化解決方案

1. 原有問題 (Line 251–305)

原本針對上、下、左、右四個方向各自撰寫獨立的 if-else 判斷，程式碼長達 55 行，且邏輯高度重複。

2. 解決方案：

透過定義座標偏移值，可將代碼精簡至約 10 行，提升可讀性與可維護性。

case 8:

```
// 定義方向偏移：0=下, 1=上, 2=左, 3=右
```

```
int[] dx = { 0, 0, -1, 1};
```

```
int[] dy = { 1, -1, 0, 0};
```

```
String[] dirName = {"下", "上", "左", "右"};
```

```
int nx = loc.x + dx[loc.dir];
```

```
int ny = loc.y + dy[loc.dir];
```

```
// 統一進行邊界與內容檢查
```

```
if (nx >= 0 && nx <= 19 && ny >= 0 && ny <= 19 && map[nx][ny] == 1) {
```

```
    object.inputMoves(press - 3);
```

```
map[nx][ny] = 0;
```

```
System.out.println("角色向" + dirName[loc.dir] + "挖");
```

```
new sounds(0);
```

```
}
```

```
break;
```

優化重點說明

1. **邏輯統一**：原本分開檢查的 $y-1$ 、 $y+1$ 、 $x-1$ 、 $x+1$ 統一由 nx 與 ny 代替。
2. **安全增強**：一次性檢查四個邊界條件，防止陣列越界。
3. **擴充性**：若未來增加斜向移動，只需在陣列中增加對應數值，不需改動主邏輯。